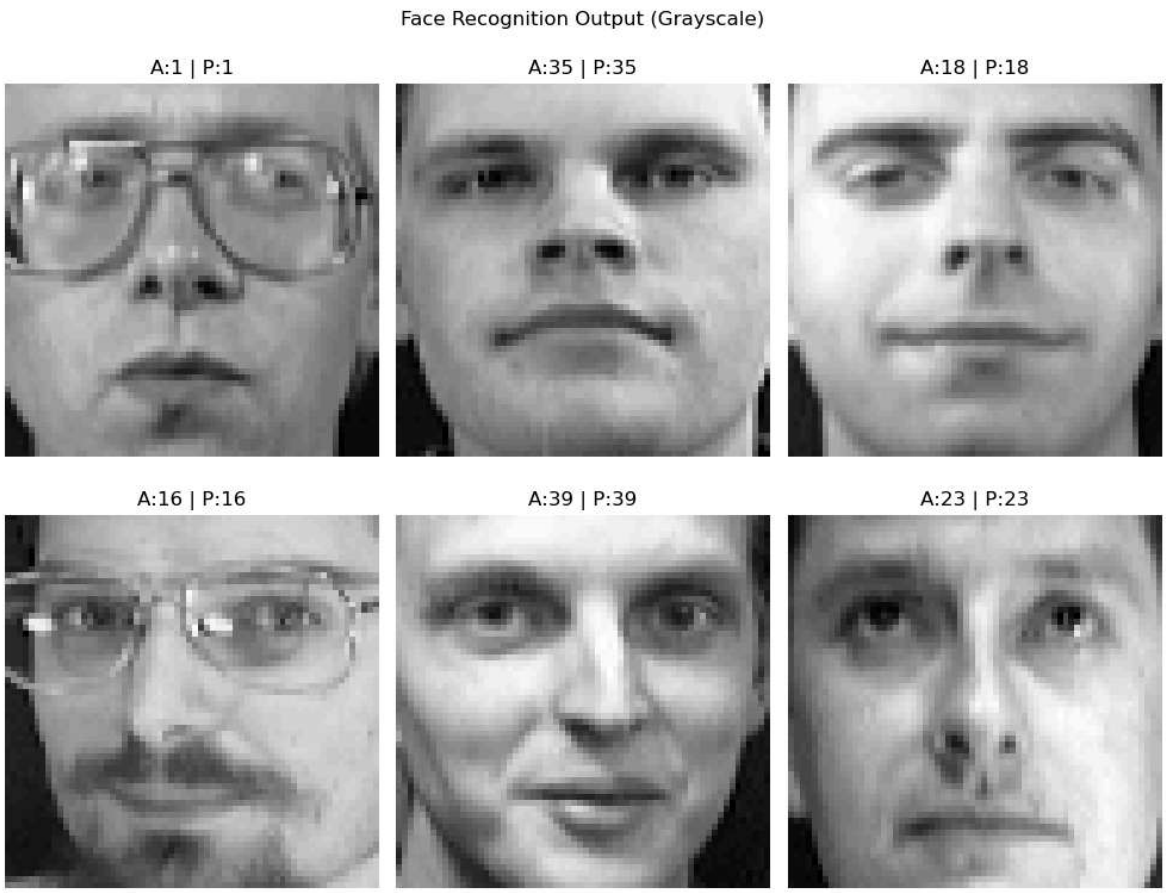
In [3]:
```
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Path to local dataset (if available)
path = r"C:\Users\mech\Desktop\PHY\olivetti_faces.npy"

# Load dataset: either from local file or sklearn
if os.path.exists(path):
    imgs = np.load(path)
    labels = np.array([i // 10 for i in range(len(imgs))])
else:
    data = fetch_olivetti_faces()
    imgs, labels = data.images, data.target

print("Dataset:", imgs.shape, "| Classes:", len(np.unique(labels)))

# Flatten grayscale images into feature vectors
features = imgs.reshape(len(imgs), -1)

print("Feature Shape:", features.shape)

# Train-test split
Xtr, Xte, Ytr, Yte, Imtr, Imte = train_test_split(
    features, labels, imgs,
    test_size=0.2, random_state=42, stratify=labels
)

# Train RandomForest
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(Xtr, Ytr)

# Predictions
pred = model.predict(Xte)
print("Accuracy:", round(accuracy_score(Yte, pred) * 100, 2), "%")

# Show sample predictions
plt.figure(figsize=(10, 8))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(Imte[i], cmap="gray")
    plt.title(f"A:{Yte[i]} | P:{pred[i]}")
    plt.axis("off")

plt.suptitle("Face Recognition Output (Grayscale)")
plt.tight_layout()
plt.show()
```

```
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to
C:\Users\omkat\scikit_learn_data
Dataset: (400, 64, 64) | Classes: 40
Feature Shape: (400, 4096)
Accuracy: 93.75 %
```


Face Recognition Output (Grayscale)
A:1 | P:1
A:35 | P:35
A:18 | P:18
A:16 | P:16
A:39 | P:39
A:23 | P:23

In [ ]:

In [ ]: